

1 The Repository

1.1 Overview

The data of the repository is used in multiple ways:

- The repository defines and describes the **interface between client and server**. The input parameters and the result sets of service requests are described.
- In case of interpreted service types, the **processing and the business logic of a service** is specified via configuration in the repository. Only in exceptional cases, an actual programming in the server is required.
- For the client, the repository defines **how the data is displayed on the client** and which GUI elements are used to enter data. The repository also defines how the client checks the user input. You can generate most of the applications on the client using the configurations of the repository. Here, programming on the client is not required.

The repository data is grouped and structured using **domains**. A domain summarizes all data that logically belongs to an application.

The domain contains hierarchically structured and typed data. A domain includes *services* and *service parameters*, the respective *GUI settings*, *properties*, *authorizations*, *ReferenceData* and *ControlDataSources*.

Find below a detailed description of the repository elements.

1.2 Domain

Domains have properties and provide services within the domain context.

A domain is the smallest software unit. You can update the domain using an update package. Create a separate domain for each application. This domain then includes the services implemented for this application. You can also use the services and client attributes of a domain in applications of other domains. For example, a client application in its own domain can use a service of a different domain.

You can assign global contents to a global domain: for example, client menu configurations or separate global syntactic types.

Name

Each domain has a unique name. For the name, you use the notation "UpperCamelCase".

1.3 Service

Services have transfer parameters and return values, which are often identical to the properties of the domains.

1.3.1 Name

Name of a service. The service name usually consists of the domain name that includes the service and the **function**, separated by a dot.

1.3.2 Function

This field describes the requested service function. Typical functions are list, update, insert, delete, new, ...

1.3.3 ServiceType

There are several service types.

InterpretedJavaService2: Services of this type are used to display lists and evaluations. The services are interpreted at runtime using repository data. Contrary to the *InterpretedJavaService*, the services of type *InterpretedJavaService2* are prepared to stream data and provide more elegant options for Java user exits.

***InterpretedJavaService* (obsolete)**: Services of this type are interpreted at runtime using repository data. These services have been replaced with the service type *InterpretedJavaService2*.

InterpretedBAPIService: You use services of this type to edit data. The services are interpreted at runtime using repository data.

ExternalJavaService: Services of this type are completely implemented in Java. You can use these services to implement lists or editing functions. You use these services if the possibilities of the interpreting service types are not sufficient and the logic must be converted into Java programming.

InterpretedWrapper: Services of this type are interpreted at runtime using the repository data. The service is implemented as wrapper of an existing PDM dialog and is therefore subject to specific limitations, e.g. it does not support any dynamic Where.

***Wrapper* (obsolete)**: Services of this type are programmed and wrap an existing BAPI function. They are therefore subject to specific limitations, e.g. no dynamic Where.

***JavaService* (obsolete)**: Services of this type are completely implemented in Java.

Recommendation:

- The type *InterpretedJavaService2* is recommended for services that you use to read data.
- The type *InterpretedBAPIService* is recommended for services that you use to write data.
- If the interpreted service types cannot meet the requirements (or only with great effort) even if they include Java user exits, you should use the services implemented in Java of type *ExternalJavaService*.
- The other service types are older technologies and should not be used for new developments.

1.3.4 ListMode

For services of type *Wrapper* or *InterpretedWrapper*: This column must be populated for each service. The column specifies whether the requested PDM dialog returns a file as result or whether it is only a return string. "Y" => The result is a file, otherwise only a string.

1.3.5 DLG

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. You fill in DLG, if **ServiceType** is *Wrapper* or *InterpretedWrapper* and if the service requests a PDM dialog with the structure "DLG=<content in this column>|..."

1.3.6 SystemCall

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. Fill in SystemCall, if the you want to run a program in the server. In the column, the name of the external program is specified. The result is a PDM dialog with the structure: "DLG=SYSTEM.CALL|PROG=<content of this column>|..."

1.4 ServiceGui

The ServiceGui data define the use and the presentation of the services on a client. You can clearly allocate the ServiceGui to a service via their name.

1.4.1 Name

The name of the service for which this data record provides presentation information.

1.4.2 Package

This field is obsolete and must be left empty.

1.4.3 Extended

This field is obsolete and must be left empty.

1.4.4 AdditionalDataLogics

This field is obsolete and must be left empty.

1.4.5 ApplicationID

Application ID used for generating applications in the client. In case of editing applications, the ApplicationID is edited with the main data source of the application that you want to generate.

1.4.6 ApplicationTitle

Language key for the title of the generated application. In case of editing applications, the ApplicationTitle is edited with the main data source of the application that you want to generate.

1.4.7 ApplicationHelpFile

File name of help file (including file extension) of the generated applications. In case of editing applications, the ApplicationHelpFile is edited with the main data source of the application that you want to generate.

The name of the help file should be independent of the technology of a used client. The client should therefore put a prefix in front of the file name. You can then design the help file displayed according to the client's technology.

Example for the client MOC: In ApplicationHelpFile, you enter "Article.pdf". The client MOC then loads the document "MOC_Article.pdf" as online help. The client automatically uses the prefix "MOC_".

1.4.8 ApplicationHelpIndex

Bookmark that is activated when Help is opened. In the main application, it is usually "Overview". You must only edit this bookmark for the main data source of the application that you want to generate.

1.4.9 Description

1.4.9.1 General

Language key for short description of service.

You can show this description on the client when the selection of services is displayed.

1.4.9.2 Processing in the MOC client

The MOC shows the description if you add a data source while configuring an application.

1.5 ServiceParameter

ServiceParameters specify the parameters of a service. They provide information on the data source and value ranges.

The service parameters include selection criteria and the columns of the result set. A service parameter can be a selection criterion or be included in the result set. The attributes described below specify if a service parameter is used as selection criterion and/or is included in the result set.

1.5.1 Acronym

Name of the parameter. The combination of **Acronym** and **ResultSet** must be unique for each service.

1.5.2 ResultSet

If the associated service returns more than one **ResultSet**, a name must be indicated here. This way, you can return results in parallel that have been calculated at the same time but have a different structure. The combination of **Acronym** and **ResultSet** must be unique for each service.

1.5.3 WebServiceType

Data type of the parameter (*decimal, integer, string, boolean, binary, datetime*). This value must be identical to the configured value of the property configuration. **IMPORTANT:** *binary* parameters are not supported by default. You can only use these parameters in user exits.

1.5.4 DefaultValue

Specifies a service default value for a parameter.

1.5.5 IsResult

Specifies whether this service parameter is part of the ResultSet (return value). If you want to use the **DefaultValue**, do not set this field (IsResult).

In case of services of type InterpretedWrapper, you must only set the column IsResult to "Y" for UPDATE, LOCK, UNLOCK, DELETE, INSERT and COPY, if the BAPI actually returns a value, e.g. a new internal_id when you create new data records.

1.5.6 IsDynamicResult

Required for the generation of the Java function (for dynamic ResultSets, the column number must automatically be extended to the fixed number). Missing columns are added as empty columns (i.e. these columns are not computed).

1.5.7 InputAsArray

The client must transfer values in form of an array. **InputAsArray** is only reasonable in case of a quantity input parameter, i.e. if at least one of the two columns, **IsSpecialParameter** and **IsFilterParameter**, is set and a quantity operator such as BETWEEN or IN is possible.

Specify if a field is an array or not (with filters always yes except for Boolean type).

If true and no array or empty, then exception. Is currently only verified in case of mandatory special parameters.

1.5.8 IsSpecialParameter

Specifies whether or not the parameter is a special type controlling the service functionality (i.e. is not a filter parameter). For the **ServiceType Wrapper**, this is the only possible parameter type. In case of the **ServiceType JavaService**, it represents a special parameter not directly included in the WHERE condition but with different "controlling" effects. If you want to use the *Default Value* on the server side, do not set this field. In addition to the defined special parameters of standard processing, you can also use other special parameters in user exits.

1.5.9 IsFilterParameter

Specifies whether it is a filter parameter. If you want to use the **DefaultValue** on the server side, do not set this field.

1.5.10 IsMandatory

Specifies whether it is a mandatory parameter for the service. If true and parameter is missing, an exception is thrown. Is currently only checked for special parameters.

1.5.11 Can* (filter) operators

This option specifies whether the service supports the relevant filter operator for this parameter. Set the "Can*" fields for filter parameters.

Available operators:

- **CanEqual**
- **CanLike**
- **CanBetween**
- **CanIn**
- **CanNotEqual**
- **CanLt (Can Less Than)**

- **CanLte (Can Less Than or Equal To)**
- **CanGt (Can Greater Than)**
- **CanGte (Can Greater Than or Equal To)**

For technical reasons, each operator has a second operator that you should select is a data record must be selected, if the operator is applicable or if the comparative value is NULL. The operator **CanEqual** will only return a data record in case of equal values, **CanEqualOrNull** in case of equal values or if the data record value is NULL. Accordingly, there are the following operators:

- **CanEqualOrNull**
- **CanLikeOrNull**
- **CanBetweenOrNull**
- **CanInOrNull**
- **CanNotEqualOrNull**
- **CanLtOrNull**
- **CanLteOrNull**
- **CanGtOrNull**
- **CanGteOrNull**

Especially with List Services you should make sure that generally all parameters support all operators in order to achieve the highest possible selectivity. In general, the framework supports this for Java services.

- You may only set **CanIn**, **CanBetween**, **CanBetweenOrNull** and **CanInOrNull**, if **InputAsArray** is also set.
- **CanLike** is only useful if the **WebServiceType** is *string*.
- With **WebServiceType** boolean, only **CanEqual** is useful.
- With **WebServiceType** string, all operators are possible.
- With all other types, all operators except for **CanLike** and **CanLikeOrNull** are useful.

Before you set wrappers, you must check which operators are actually supported by the PDM dialog or the system command.

1.5.12 HydraAcronym

With service type *InterpretedWrapper*, the HYDRA acronym is specified.

1.5.13 HydraResultAcronym

If the acronym of the selection criterion is different to the acronym in the result file, you can enter an acronym that is different to the HydraAcronym for the service type *InterpretedWrapper* and *ListMode=Y*.

1.5.14 TransferEmptyValuesToHydra

Specifies whether blank values, too, are to be transferred to the server, or whether the ID is simply omitted. "Y" => blank values are transferred, otherwise => ID is completely omitted.

Note: You must set this field for Insert and Update (editing screens). Only then, you can enter blank values and/or overwrite existing values with blank values.

1.5.15 HydraShiftPart

The following components are combined with the **Reference** field: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. These components are marked as belonging together. The column "HydraShiftPart" can include the following values:

- beginDate
- beginTime
- beginDatetime
- endTime
- endDatetime

Important: The column can only be populated if the parameter is part of a group that includes the following five components: Start of shift date, start of shift time, end of shift, end of shift time stamp, start of shift time stamp. The column must not be populated if it is only a group of three components including date, time and date + time field. In this case, ONLY populate the **Reference** column.

1.5.16 Reference

Is used to generate a DateTime data type from one field each for the date and the time (in seconds after midnight) and to identify the shift parameters.

1.5.17 TransformationType

Use this field to specify transformations for input and result parameters for List Services/wrappers (e.g. convert Bool to J/N and vice-versa or correct filtering for DateTime fields that consist of two fields in the database). For further details on this field, refer to section 1.10.

1.5.18 PlugName

Specifies whether the result parameter for this service is directly derived from the specified DataObject or whether it is added to the DataObject via plug.

Example:

Service A.List uses a plug of service B.List in the service parameter b. Consequently, the following configuration applies to service A.List:

ServiceParameter	DataObjectName	PlugName
a	A.List	
b	A.List	B.List

If the field *PlugName* includes a value, the Interpreter replaces the values of the *ServiceParameter* with those values of the plugged service when creating the SQL statement.

In the special case where an interpreted List Service does not use an own table but only plugs, and subsequently adds fields via user exit, these fields should state USEREXIT!



If you create new services, it is recommended to avoid plugs and to provide data directly via the DataObject via Join. Dependencies between several services are thus avoided.

1.5.19 DBField

Database field that you use to make a selection. Write the database field in lower case. You can either enter simply the field name or (for complex expressions) the expression with placeholders for the alias (e.g. `hydadm.get_datetime(%1$s.bearb_date,%1$s.bearb_time)` or `{fn substring(%1$s.field,2,1)}`).

Proceed as follows for joins to other tables:

Entry: <ALIAS>.<DBfield>

Example:

DB field: STA1.status_bez

Acronym: gage.status.designation

Table: caq_status (STA1)

Conditions: status_typ = 'PMSTATUS', status_nr = status

1.5.20 DBAlias

The alias for the table that is used to select the value for the acronym.

1.5.21 DBTabelle

The table that is used to select the value for the acronym.

1.5.22 DBFieldAlternative

If you cannot use the **DBField** because the **ConditionalFieldKey** is not applicable, you use the **DBFieldAlternative**.

You can enter a number, 'null', 'string', {fn ...} or another field / subselect. If it is another field or subselect, you MUST enter %1\$s for the alias of the table.

If **DBFieldAlternative** is empty, but you require an alternative field, NULL is selected.

1.5.23 DataObjectName

If a service uses several data sources to identify its data, you can store the data source (= DataObject = DO) that issues the result parameter in this field. For example: A service includes the parameters a, b and c:

- a is computed,
- b is identified using data object (DO) F and
- c is identified using data object (DO) G.

For a: the field is blank. For b: the field contains F. For c: the field contains G. Is used as reference for the ...do.xml configuration.

1.5.24 ConditionalFieldKey

This field specifies if a DB field is only conditionally available. The ConfigurationManager checks the condition for the existence of the field. Enter the feature key of the Configuration Manager (**feature set**) in this repository field to enable the check.



If a parameter is a conditional field and the condition is not fulfilled, the entries for the MOC acronym are removed from the ComplexSelectMap and the SpecialFilterMap.

As a result, the changes in the Special Filter Map via user exits and transformation type are also lost!

1.5.25 Constraints

Constraints are processing parameters that are used for **ServiceType** *InterpretedBAPIService*. Constraints are structured as keys with optional values. The separator between keys is the pipe character (|). You use a semicolon to separate various values. You use the equal sign (=) to separate key and value. The general structure is as follows:

Key1=Value;Value;Value|Key2|Key3=Value|

The following constraints are available:

Constraint Key	Constraint value(s)	Description
KEY	exactly one number between 1 and 5	Define field as key including key number for hyd_lock table
SERIAL	none	Field is a SERIAL (and/or auto-increment)
SEP_DATETIME	1st parameter refers to the date field 2nd parameter refers to the time field	Allows processing of separate date and time fields
BOOL	1st parameter is the value to be entered into the DB if true 2nd parameter is the value to be entered into the DB if false 3rd parameter is the value to be entered into the DB if null (null for null) 4th parameter is the type of DB field, e.g. BOOL=J;N>null string BOOL=1;0>null;integer	Use this to write Boolean values into a string or Integer Field.
MODIFY_TS	None	
MODIFY_BY	None	
CREATE_TS	None	
CREATE_BY	None	

1.6 ServiceParameterGui

The ServiceParameterGui define how ServiceParameters are displayed on the client. Use **Acronym** and **ResultSet** to clearly allocate ServiceParameterGui to a service parameter.

A number of settings exist in the `ServiceParameterGui` and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the `ServiceParameterGui` if you want to display specific services on the client in a way that is different to the settings in the properties. The `ServiceParameterGui` fields overwrite the property fields of the same name.

1.6.1 Acronym

Name of the parameter for which this data record provides presentation information. There must be a corresponding property for each **acronym** of a parameter.

1.6.2 ResultSet

See **ResultSet** with `ServiceParameter`.

1.6.3 Label

1.6.3.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

1.6.3.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

1.6.4 Tooltip

Specifies a specific tooltip for the parameter in the service context. Entry as language key.

1.6.5 FormatType

Use this field to overwrite specific values of a property in relation to the service (currently **Label**, **Length**, **ControlType**, **ControlTypeMode**, **ControlDataSource**, **ControlDataSourceMode**, **ControlDataSourceResult**).

For example: If you enter *workplace.id* as **FormatType** for the parameter *resource.id*, you can define for the parameter to be a *resource.id* in this service, however its length, label and control properties are taken from *workplace.id*.

In this case (other than in case of semantic and syntactic types), the value from **FormatType** takes priority. For this reason, we have a new hierarchy:

- Value from **FormatType**
- Value from ServiceParameterGUI
- Value from Property
- Value from **SemanticType**
- Value from **SyntacticType**

1.6.6 ClientDefaultValue

Input fields have a **ClientDefaultValue** property. The value entered here is displayed as default value when the control is initialized. "From" and "to" values are separated by semicolons.

Set checkbox: If the value of this field is set to *true* during a CheckEdit, the checkbox is set after initializing.

Preallocation of text fields with "from" and "to" values (InputAsArray): set value1;value2 to prepopulate the 'from' and 'to' fields during a text edit.

Date fields: In case of date fields, the field can be preallocated with an offset. If you set default values for date fields, you must absolutely specify the type of offset. The following offsets are possible:

- h (hours)
- d (days)
- w (weeks)
- m (months)
- y (years)

The 'to' value is always **relative to the 'from' value**. The default value is always a DateTime object. The presentation depends on the output format of the relevant field.

You can put "[" and "]" in front and at the end of the relevant value to specify the start and end of a period of time. Consequently, e.g. "[0d;0d]" means that 12:00:00 AM is entered in the 'from' field today and 11:59:59 PM is entered in the 'to' field today. "[-1w;0w]" means from Monday last week up to Sunday last week.

Examples

Current date:

0d

From today to the day after tomorrow:

0d;2d

From today to one week from today:

0d;1w

From yesterday to tomorrow:

-1d;2d

From one year ago today to one year from today:

-1y;2y

Year shortlists: You can configure a year shortlist by **ControlDataSource** = YearList and **ControlDataSourceMode** = Script, or even by standard "Service-ControlDataSource". In this case, you can use the following default values:

- Current year: 0y and/or currentyear
- Last year: -1y
- Following year: 1y
- 4 years ago: -4y
- Year that was current 10 months ago: y-10m → this is mostly the case when the relevant year field is used in combination with a month shortlist.

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y

Month shortlists: You can use the following default values for a month shortlist:

- Current month: 0m
- Last month: -1m
- Following month: 1m
- 4 months ago: -4m

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y.

1.6.7 IsKey

The IsKey column is very important and should be occupied for all key columns of a service, since otherwise data records cannot be clearly identified. Columns including the value 'null' may NOT be defined as keys. The IsKey columns should be identical for all services (insert, update, delete, lock, unlock, copy). These entries are important, so it is best to verify them twice.

This field specifies the positioning of the cursor after an editing operation. If the positioning option OnKeyValue is selected, the client should only request one new data row after editing. You also use values that are marked **IsKey** as selection criteria. **IsKey** must also be set for *delete*, since this data record must be deleted from the view.

IsKey must also be indicated for *list*. If no sorting is given in the *list*, sorting takes place according to **IsKey** fields.

Every parameter which is **IsKey** MUST always be **IsMandatory**. This rule has two exceptions:

- List service
- Wrappers with composed keys.

1.6.8 ShowInGrid

Specifies whether the parameter is to be displayed in tables by default.

1.6.9 ShowInDetail

Specifies whether the parameter is to be displayed in detail views by default.

1.6.10 ShowInSearch

Specifies if the parameter is to be used as selection criterion (i.e. in selection panels) by default.

1.6.11 ColumnCategory

1.6.11.1 General

In the tabular view, the client should provide the option to summarize the columns in the table to categories. You specify a language key that is displayed as title of the summarized columns.

1.6.11.2 Processing in the MOC client

The ColumnCategory is used to assign the parameter to a "strip" in the grid (table view).

1.6.12 Category1, Category2, Category3

1.6.12.1 General

The client processes the columns Category1, Category2, Category3 in order to group fields in applications. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

1.6.12.2 Processing in the MOC client

Category1: Assigns the parameter to a tab in the detail view.

Category2: Grouping options for detail screens.

Category3: Currently not used.

1.6.13 TabOrder

You specify the order of tabs for detail views.

1.6.14 ColumnOrder

You specify the order of columns in tabular views.

1.6.15 ShowSecondControllnSearch

1.6.15.1 General

Specifies whether a second control is to be displayed (from/to). You can use this setting with selection criteria that include a value range via the operator CanBetween, e.g. "date from/to".

1.6.15.2 Processing in the MOC client

The MOC provides two adjoining fields. The label text of the second field is automatically "to". If it is a field of "date" type, you can predefine a relative date for both fields.

1.6.16 SearchTabOrder

Specifies the tab sequence for the selection panel.

1.6.17 SearchCategory1, SearchCategory2

1.6.17.1 General

The client processes the columns SearchCategory1 and SearchCategory2 in order to group fields in selection panels. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

1.6.17.2 Processing in the MOC client

SearchCategory1: You allocate the parameter to a tab in the selection panel.

SearchCategory2: Grouping options for the selection panel.

1.6.18 ControlType

Use the ControlType to specify which control should be used for the relevant parameter. The client will map the abstract type onto a specific control class. If you do not specify a type, the client uses the data type to decide on the ControlType. Possible values for the ControlType:

CheckEdit: Selects a Boolean value (true/false) or multiple values if a reference to a data source is given.

ColorEdit: Selects a color value.

ComboBoxEdit: Combobox with selection of values from web service or data reference.

DateTimeEdit: Enter a date and/or a time.

MemoEdit: Enter an arbitrary text.

RadioGroup: Selects a Boolean value (true/false) or one of multiple values if a reference to a data source is given.

TextEdit: Standard text input. You can add a button opening a search dialog to this control, if you add a reference to a service in **ControlDataSource**. If you enter the name of a DataLogic in **ControlParameter** and if a mapping is included in **ControlDataSourceResult**, data will be requested upon leaving the control and return values will be mapped appropriately.

1.6.19 ControlTypeMode

1.6.19.1 General

Allows for controlling the input control.

CheckEdit: DualState (default), TriState, J;N;J (checked;unchecked;tristate)

ColorEdit: none

ComboBoxEdit: SingleEdit, Single, Multiple (multiple selection)

DateTimeEdit: Date (date display), Time (time display), DateTime, RelativeDate, RelativeDateTime

MemoEdit: none .

RadioGroup: SingleColumn, SingleRow

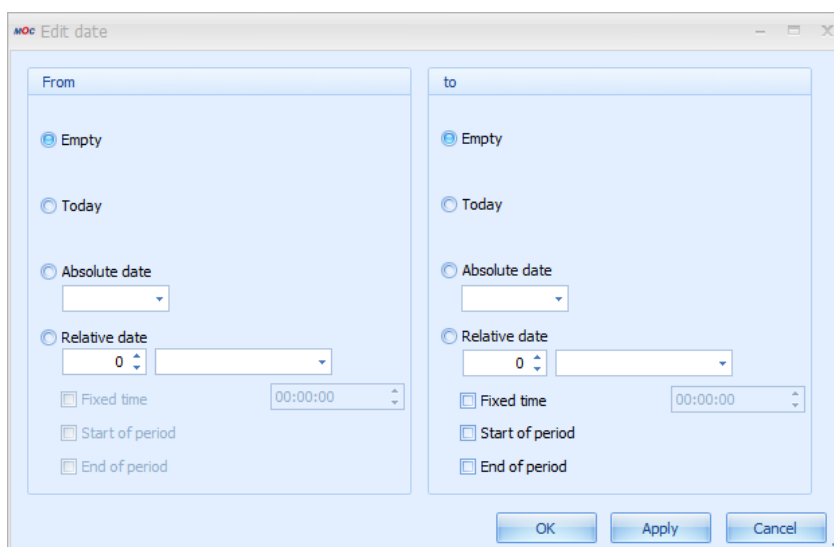
TextEdit:

- Empty: the search button is shown if a **ControlDataSource** is defined.
- "SearchButton": Search button is shown.
- "SearchButtonValidate": Search button is shown. If you enter an invalid value, an error is displayed.
- "OpenFileDialog": opens a file selection dialog.

1.6.19.2 Processing in the MOC client

If you use **DateTimeEdit** including the definition of a relative date (**ControlTypeMode:** RelativeDate or RelativeDateTime), you can enter a relative date.

If **ShowSecondControl** = true, you can predefine the complete relative value range. In this case, a button is displayed behind the second input control. You can use this button to open the following dialog:



Use this dialog to customize the values for **ClientDefaultValue** . The following entries are possible:

- **Empty:** no value is adopted

- **Today:** the current date is adopted
- **Absolute date:** you can select a fixed date value via a calendar control
- **Relative date:** you can select and adopt a date relative to the current date. In this context, "Start of period" means that you additionally go to the start of the selected period. Example: current date is 20-MAY-2010. If you select "- 1 month", 20-APR-2010 is adopted. If you also select "Start of period", the date is changed to 01-APR-2010. The same applies to "End of period". These settings are saved in the mpdvEdit or the selection profiles as **ClientDefaultValue**.

1.6.20 ControlParameter

See **ControlType** → **TextEdit**

1.6.21 ControlDataSource

Data source for the selection of values. The data source can be:

- **Web service** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = **Name** of a **ControlDataSource**. See also section "1.8 ControlDataSource")
- **ReferenceData** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Reference*, **ControlDataSource** = **Type** of **ReferenceData**)
- **Search application** (configuration: **ControlType** = *TextEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = application name)
- **Script** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Script*, **ControlDataSource** = Name of script)

1.6.22 ControlDataSourceMode

Data source mode (*Lookup*, *Reference* or *Script*).

1.6.23 ControlDataSourceParameter

Optional setting of parameters of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

See also the description **ControlDataSource - Parameter**

1.6.24 ControlDataSourceResult

Optional setting of the result of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

The settings in this field provide more options than the **Result** in the **ControlDataSource**:

Result columns are separated by semicolon. Field mapping:

- First entry: Value
- Second entry: Labeling
- Third entry: UnitLabel
- As of the fourth entry, the fields are mapped:
 - o Via acronym or semantic type.
 - o Field mapping: in **ControlDataSourceResult**, you can enter a mapping in the form "FieldName=ColumnFromResult" as from the fourth entry. For example, you can specify tool.id=resource.id in order to fill the field "tools.id" with the "resource.id" value from the search application. Several mappings are separated by ";" - spaces are not allowed.
 - o Asterisk mapping: Instead of mapping, you can also enter * . Subsequently, all return columns of the search application are mapped. The mapping is performed as usual via ID or semantic type.

1.6.25 VisibleCondition

This value decides whether an input field is visible on the client. For customization, see EditableCondition.

1.6.26 EditableCondition

This value decides whether you can edit an input field on the client. There are three possibilities:

- Boolean value: In case of *TRUE* or *FALSE*, the field is always editable / non-editable.
- Binary expression:
 - o Field name must be the name of a field that is also located in the ControlPanel.
 - o Valid operators: =, <, >, <=, >=, <>, !=
 - o The value is written as a string and interpreted depending on the comparative field value.
 - o Field, operator and value must be separated by a space!
- Concatenation of binary expressions:
 - o You can concatenate an arbitrary number of binary expressions.
 - o You can use the operators "&&", "AND", "||", "OR" to link expressions.
 - o Here, too, all components of the conditions must be separated by a space.
 - o Priority of operators: "AND" or "&&" are evaluated first, then "OR" and "||".
You cannot use brackets.
 - o Example: resource.id = 12345 && resource.costcenter = 20 || resource.id = 60610



The client assigns the default value of the property "ClientDefaultValue" to the field, if the result of an expression in the EditableCondition or the VisibleCondition changes from FALSE to TRUE. The client dynamically evaluates the expressions in the EditableCondition and the VisibleCondition, if the fields of the application change.

1.6.27 ScriptId

1.6.27.1 General

The ID of the script that is allocated to the parameter. If you set the ID, the relevant script is performed upon various events (at present EditValueChanged and Leave).

1.6.27.2 Processing in the MOC client

The method name of the script is ScriptId+EditValueChanged and/or ScriptId+Leave. The script can be included in any DLL that is read by the CodeManager.

1.7 Property

For the acronyms, properties include information on data types, input and output formats, display options, a name (that can be localized) and other settings specifying how ServiceParameters are displayed in the client. Each property has a system-wide unique acronym.

A number of settings exist in the ServiceParameterGui and in the properties. You normally use the properties to define how data is displayed on the client. You only fill the respective field in the ServiceParameterGui if you want to display specific services on the client in a way that is different to the settings in the properties. The ServiceParameterGui fields overwrite the property fields of the same name.

1.7.1 Acronym

Clear identification of the property across all domains.

1.7.2 WebServiceType

Describes the data type used to transfer the property between client and server. The currently supported WebServiceTypes are exclusively

- *binary*
- *boolean*
- *datetime*
- *decimal*
- *integer*

- string

Important: the types **date* and **time* are internal types which are not transferred.

1.7.3 NETType

The data type used by the client. If NETType is empty, the **WebServiceType** is used to automatically identify the data type used by the client. At present, NETType supports the following entries:

- **color:** Use *color* to convert the transferred integer into an RGB code. In this case, the conversion is implemented by the grid.
- **duration:** creates a duration from an integer.
- **image:** either creates an image from a transferred byte array or interprets a transferred string as image name. For example, the *maintenance.active.led* property is transferred as a string including the name of an icon.
- **preview:** Specifies that the contents in the client may be displayed as "preview" (similar to auto-preview outlook) (application e.g. in DevExpress grid).
- **timestamp:** Use *timestamp* to automatically create an additional column for date values in the client in order to process time and date separately.

1.7.4 SemanticType

Use semantic types to inherit semantic properties. The "order.id" is therefore used to identify orders (semantic meaning). The acronym *operation.order.id* includes such an order identification and therefore has the semantic type *order.id*. If an attribute of the property is not set (empty), the respective value from the semantic type is used for the processing in the client.

For example: You must set the semantic type if you want to adopt a value from a lookup screen in the field. For the *workplace* field, enter e.g. *resource.id* as semantic type in order to adopt the selected workplace from a search screen for workplaces. Refer to the description of the SyntacticType for further information on the priority used to specify the attributes of a Property, the SemanticType and the SyntacticType.

1.7.5 SyntacticType

You mainly use a syntactic type for a uniform presentation of the different properties. The syntactic type does not include any semantic content. For example: The properties *booking.begin_ts* and *booking.shift.start_ts* have different semantic meanings, but are presented in a uniform format that can be controlled centrally.

Syntactic types are used to control the characteristics of a *Property*: for example length, input and output screen, tooltip, label, etc. To select the valid value for a characteristic, the client proceeds as follows:

- If the characteristic (e.g. length) is set in Property, the client uses this value.
- Or: If a semantic type is available and the characteristic is set, the client uses this value.
- Or: If a syntactic type is available and the characteristic is set, the client uses this value.

Note:

- You must always enter a **description** for syntactic and semantic types.
- Syntactic types can reference other syntactic types so that "inheritance hierarchies" can be created.
- Create syntactic types as property of the *SyntacticType* domain.
- Semantic types are usually "real" properties of a "normal" domain that are used as semantic type at other places.

1.7.6 Label

1.7.6.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

1.7.6.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

1.7.7 DefaultTooltip

Specifies the default tooltip for the property as language key.

1.7.8 UnitLabel

Text key for unit. The unit is displayed to the right of the input field.

1.7.9 OutputFormat

This field specifies the format that is used to display a value (e.g. for date or quantity values). If you do not enter an **InputFormat** in the repository, the MOC tries to develop an appropriate format from the **OutputFormat**. Enter the value **InputFormat** in the repository only if special masking is required. Find further details in section "1.7.12 Rules for the input/output formatting".

1.7.10 InputFormat

Equivalent to OutputFormat. You can enter a valid regular expression in the field InputFormat. Other entries that are not regular expressions are not permissible. Find further details in section 1.7.12.

1.7.11 Length

The client shows the control for this acronym in the specified width (i.e. the specified number of characters). With Length=0, the control uses the entire width available. If a width is specified but the space available is not sufficient, the control is cut off.

This field also specifies the number of characters that you can enter in an input field with **ControlType = TextEdit**, if no other **InputFormat** is specified.

1.7.12 Rules for the input/output formatting

Overview

In the repository, you define the formatting of the data output and the input dialogs to edit data. The "Properties" of the different acronyms include an **OutputFormat** and an **InputFormat**. The **OutputFormat** defines formatting if you display a value.

Important: If you do not enter an **InputFormat** in the repository, the MOC uses the **OutputFormat** to generate an appropriate formatting. Enter the value InputFormat in the repository only if special masking is required.



In case of strings, you cannot enter the special characters asterisk (*) and pipe (|), if you have not defined any input format. As you use these two special characters as separator and control character, they can cause problems if they are written in the database.



With strings, the maximum number of characters that you can enter is defined by the attribute **Length**, if no other input format is defined.

Syntactic types

The Properties provide so-called "syntactic types" in order to make groups (similar to field types in Delphi). Syntactic types have the same properties as real properties. The real properties have a syntactic type. For example, if the **output format** of the syntactic type includes a value, this value is used wherever this syntactic type is entered.

Example: Industrial minutes

The syntactic type "Durations" has the format {0:mpdv_timespan}. With the different properties showing durations, "Durations" is entered in the column **SyntacticType** and no entries are made in the columns "output format" and "input format". When the property is read - and if no **output format** is available in the property - the format of the syntactic type is used.

If a system displays industrial minutes (no standard function!) and if the syntactic type "Durations" is specified, the **output format** is automatically changed from {0:mpdv_timespan} to {0:mpdv_industrialMinutes}. As a result, all formats including the syntactic type "Durations" are shown in industrial time units.

Times and durations are internally stored in the system as integer seconds. If you convert times or durations during input or output formatting to formats other than hours, minutes and seconds (HH:MM:SS), the conversion may not be possible without losses. For example, this applies to the use of the "mpdv_calc" format and the classic industry minute display:



When converting from seconds to hours (division by 3600), decimal numbers with an infinite number of decimal places can occur, which inevitably have to be rounded when displayed on the client. Example: 20 minutes = 1200 seconds = 0.333333... hours. If the value is rounded to three decimal places, you calculate backward as follows: $0.333 \times 3600 = 1198.8$ seconds. Depending on how the client rounds, the internal value is then no longer 1200 seconds, but 1999 or 1998 seconds.

If you use less than three decimal places, the conversion error gets even greater:

The system recorded a duration of 123 seconds. The client displays 0.03 hours. If you calculate backwards, the result is 108 seconds.

Output formats

OutputFormat	Automatically created masking (input format)	Examples	Description
Numeric data			
f(number)	None	f3, f1	Numeric value without thousands separator. The number specifies the number of decimal places.
n(number)	None	n0, n2, n5	Numeric value with thousands separator. The number specifies the number of decimal places, even if the data type to be displayed is an integer type. In case of n0, no decimal places will be shown.
#. (##) , #. (0)	None	#.####, #.0000	Arbitrary format
{0:mpdv_timespan}	None	2:33:30	MPDV format provider. Conversion of seconds to hh:mm:ss and vice-versa.

{0:mpdv_timespan_short}	None	2:33	MPDV format provider. Conversion of seconds to hh:mm and vice-versa.
{0:mpdv_timespan_minutes}	None	45	MPDV format provider. Conversion of seconds to minutes and vice-versa.
{0:mpdv_cycletime}	None	1:30:00	MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa.
{0:mpdv_te}	None	2.00	MPDV format provider. Hours per 1000 pieces. Conversion into seconds and vice versa.
Strings			
empty	[^*]]*		Illegal characters begin with ^. In this example * and * No limitation in length
empty	[^*]]{0.10}		Illegal characters begin with ^. Max. length: 10 characters
empty	[0-9a-fA-F]		Allowed characters 0 through 9, a through f, A through F.
Special formats			
{0:mpdv_cycletime_sec_cycle}	None	29 sec/cycle	MPDV format provider. Seconds per cycle. Conversion into seconds per 1000.
{0:mpdv_IndustrialMinutes}	None	1.50	MPDV format provider. Conversion of seconds into industrial minutes and vice versa.
{0:mpdv_leadingzeros_order}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "order_id". The basic settings are used to automatically specify the length.
{0:mpdv_leadingzeros_operation}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "operation". The basic settings are used to automatically specify the length.
{0:mpdv_leadingzeros_sequence}	ORDER		You must combine this output format with the input format ORDER. The combination is used in the syntactic type "ordersequence_id". The basic settings are used to automatically specify the length.

Input formats

The following definitions are available for the input format:

- Leave empty: The input format is implicitly defined using the output format. See table above.
- Use of logical input formats
- Use of regular expressions

Logical input formats

To simplify the definition of input formats and limit the variety of entries in the repository, the logical input formats are provided. These input formats are permanently implemented in the client and can directly be used in the repository. Input formats are customized in the properties. But service parameters specify whether wildcards are allowed. For this reason, the input format actually used can vary depending on the allocated service.

In order to use logical input formats, define the name of the input format in the affected property in the repository. The following formats are currently available:

Name	Input format without wildcard	Input format with wildcard
CHARACTER	[^*][LENGTH]	[^][LENGTH]
NUMBER_N0	[0-9][LENGTH]	[0-9][LENGTH]
NUMBER_N1	[0-9][LENGTH]\R.[0-9]{0,1}	[0-9][LENGTH]\R.[0-9]{0,1}
NUMBER_N2	[0-9][LENGTH]\R.[0-9]{0,2}	[0-9][LENGTH]\R.[0-9]{0,2}
NUMBER_N3	[0-9][LENGTH]\R.[0-9]{0,3}	[0-9][LENGTH]\R.[0-9]{0,3}
NUMBER_N6	[0-9][LENGTH]\R.[0-9]{0,6}	[0-9][LENGTH]\R.[0-9]{0,6}
TIMESPAN_SHORT	[0-9][LENGTH]\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}
TIMESPAN	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}
ORDER	[0-9a-zA-Z.+][LENGTH]	[0-9a-zA-Z.+*][LENGTH]

The placeholder [LENGTH] is replaced with the configured field length at runtime. If the defined length is '0', an '*' is entered. With the logical format "ORDER", the system automatically changes the [LENGTH] according to the basic settings when the output format changes.

Input/Output formats including calculation

If you specify the **output format** *mpdv_calc*, you can include calculations in the formatting. In the format, you can specify a divisor and multiplier and an identifier that specifies if a reciprocal is calculated. You can also specify the number of decimal places. The **OutputFormat** *mpdv_calc* implicitly defines the **InputFormat**. If an input of values is made, the reciprocal value is calculated.

Example: "mpdv_calc;MULT=5;DIV=2;INVERSE=false;FORMAT=n3" (the value is multiplied by 5, divided by 2, then the reciprocal is calculated and the result is displayed with 3 decimal places).

The input/output format including calculation is normally used for the display of cycle times or specifications of single pieces. In the database, these times are always saved in seconds per 1000 pieces. If an input/output format including calculation is used, you can convert the times to hours per 1000 pieces, minutes per piece or with reciprocal also to piece per hour.

Overview of regular expressions

You can find a large amount of information on regular expressions using the search engines on the internet. In the following, the most important aspects are presented.

Meta characters

Represent a range of characters.

Character	Description
.	Matches any character.
[<i>aeiou</i>]	Matches any single character included in the specified set of characters.
[^ <i>aeiou</i>]	Matches any single character, which is not included in the specified set of characters.
[<i>0-9a-fA-F</i>]	Use of a hyphen (–) allows specification of contiguous character ranges.
\R.	Matches the decimal separator specified by the System.Globalization.NumberFormatInfo.NumberDecimalSeparator property of the current culture.
\R:	Matches the time separator specified by the DateTimeFormatInfo.TimeSeparator property of the current culture.

Quantifier

Repetition, number of characters

Quantifier	Description	Samples
*	Specifies zero or more matches.	The "\w*" mask matches a string consisting of zero or more letter characters. It's equivalent to the "\w{0,}" mask.
+	Specifies one or more matches.	The "\w+" mask matches a string consisting of one or more letter characters. It's equivalent to the "\w{1,}" mask.
?	Specifies zero or one match.	The "\w?" mask matches zero or one letter character. It's equivalent to the "\w{0,1}" mask.
{ <i>n</i> }	Specifies exactly <i>n</i> matches.	The "\d{4}" mask matches exactly four digits.
{ <i>n</i> ,}	Specifies at least <i>n</i> matches.	The "\d{2,}" mask matches two or more digits.
{ <i>n</i> , <i>m</i> }	Specifies at least <i>n</i> , but no more than <i>m</i> , matches.	The "\d{1,3}" mask matches either one, or two, or three digits.

Special characters

Special characters

Character	Description	Samples
	Alternation symbol. This can be used to implement a choice between two or more alternatives.	The "1 2 3" mask matches either "1" or "2" or "3". The "abc 123" mask matches either "abc" or "123". The "\d{2} \p{L}{2}" mask matches either two digits or two letters.
()	Grouping. You can use parentheses to create sub-expressions, or to limit the scope of the alternation.	The "(an ba)t" mask matches either "ant" or "bat". The "(net)+" mask matches "net", "netnet", "netnetnet", ... strings. Compare with the "net+" mask which matches the "net", "nett", "nettt", ... strings. The "(0 1)+" mask matches a string of indeterminate length, consisting of "0" and "1".

Examples

Input 1..9999 => Input format for property : ([1-9][1-9][0-9][1-9][0-9][0-9][1-9][0-9][0-9][0-9])

Input 0..999 => Input format for property : ([0-9][1-9][0-9])100)

Best practice: input of long string fields

The client identifies the width of an input field using the attribute **Length**. In case of long string fields with more than 20 characters, the layout can become confusing because these string fields use the complete width of the layout and are very long compared to other input fields. Very long string fields are cut off on the right-hand side, if the available space is not enough. To avoid this behavior, you can control the displayed field width regardless of the number of characters that you can enter.

- Use the attribute **Length** to specify the **width** of the input field.
- You can use a regular expression in the **InputFormat** to specify the **number of characters that you can enter**.

If you enter strings that are larger than the displayed field, the input field automatically scrolls horizontally.

Examples:

Attribute	Length	InputFormat	Effect
article.designation	50	.{0.250}	The field is displayed with a width of 50 characters. You can enter up to 250 characters.
operation.input_component_list	25	.{0.40}	The field is displayed with a width of 25 characters. You can enter up to 40 characters.

1.7.13 FillChar

Obsolete. This field must be left empty.

1.7.14 Calculation

Obsolete. This field must be left empty.

1.7.15 Further fields see ServiceParameterGui

For a description of the following fields, refer to the data types of the ServiceParameterGui:

ControlType, ControlTypeMode, ControlParameter, ControlDataSource, ControlDataSourceMode, ControlDataSourceParameter, ControlDataSourceResult, VisibleCondition and EditableCondition.

1.8 ControlDataSource

A ControlDataSource defines a data source that you can use to fill selection lists in controls, for example. These can be data logics (service requests) or reference values (see also ReferenceData).

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents.

You use data logics to request services that identify selection lists (or RadioGroups) dynamically. For example, these lists can include master data that are configured in the database.

The settings made in the columns **Parameter** and **Result** can be overwritten in a **Property** or **ServiceParameterGui**.

1.8.1 Name

Name of the ControlDataSource. The name should be composed of English terms clearly describing the data source. You usually use the camelCase notation.

1.8.2 Source

If the data source is a web service, this field contains the name of the client's data logic. You derive the data logic from the service name. To do so, remove the dot between domain and function and use a capital letter for the first letter of the function:

Service	Data logic
MDUser.list	MDUserList
MDUnits.list	MDUnitsList

In case of reference values, this field includes the **Type** of a **ReferenceData**.

1.8.3 Parameter

A list of parameters. The list does not include spaces, use semicolons to separate parameters. This field is only allowed in combination with web service data sources. A parameter can be allocated dynamically or permanently.

Permanent parameters appear as <acronym>=<value>, e.g.

"dialogconfiguration.type=AIPDEF;dialogconfiguration.type=AIPTNR".

Dynamic parameters are specified as a pair of <acronym1>=[<acronym2>]. e.g.

"resource.id=[resource.id];pdvprocessparameter.evaluation_ts=[pdvsinglevalue.evaluation_ts]"

The acronym in square brackets is replaced with the acronym values from the ControlPanel.

1.8.4 Columns

A list of requested columns. The list does not include spaces. To separate columns, semicolons are used. This is only permissible for web service data sources.

1.8.5 Result

You can enter 1-n acronyms separated by semicolon. The sequence used specifies the importance.

- Position 1 (Value): Name of acronym whose value is entered in the input field.
- Position 2 (ControlValue): Name of acronym whose value is displayed in the selection list. If you do not specify position 2, the acronym of position will be displayed.
- Position 3 (LabelValue): If you specify position 3, the value of the acronym is entered in the label field of the input field and also displayed in the selection list.
- Position 4-n: Use these positions to define additional return values, which are then used to update "dependent" controls in the client ("lookup").

Only with web service data sources:

Optional return columns of the data source, separated by semicolons. Without spaces. The return has the format <acronym>=<value> - for acronym pairs, the second acronym is therefore replaced with the result value (e.g. if you enter "operation.resource.id=resource.id", this results in "operation.resource.id=4711").

1.9 ReferenceData

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents. In contrast to values provided by web services, reference values are fixed and do not change. For this reason, reference values can be entered once in a list and are delivered in this form.

1.9.1 ref_data_key

The **ref_data_key** must be unambiguous for each entry. In special cases, this key is used in the source code (at least in the server).

Usually, the **ref_data_key** is composed of **type** + : + **db_key**; this facilitates its allocation to type and key. An exception occurs if the **db_key** includes a German expression. The **ref_data_key** must then be formed differently. For example, *pwdexclusion:person.firstname* is a super **ref_data_key** for the type *pwdexclusion.pwd* and **db_key** *PNR.PVORNAME*.

1.9.2 Type

Use this field to summarize various ReferenceData entries to a list.

1.9.3 db_key

The **db_key** is the actual value that is selected in the list. This key identifies an entry unambiguously within a **Type**. You cannot freely select the key because the key is often transferred to services and can correspond to the content of a configuration identifier in the database, for example.

1.9.4 is_default

The entry with this key is preallocated as default.

1.9.5 Designation

Text displayed in the selection list. A language key is specified.

1.9.6 sort_key

Specifies the sequence that is used to display the entries in the selection list.

1.10 Authorization

The authorization mechanism

- protects applications and functions against unauthorized use on the client,
- hides fields or field groups on the GUI,
- prevents these fields from being edited.

1.10.1 Authorization type

Controls the type of authorization. Possible values:

- Acronym: enables the authorization of individual fields (properties)
- AcronymGroups: enables the authorization to group fields
- Application: enables the authorization of applications
- Functions: enables the authorization of functions which are e.g. requested from the application toolbar.

1.10.2 Authorization Context

Context where the authorization is intended. If the field is left empty, authorization is always granted, irrespective of the context. You normally use this field to control the authorization of acronyms in the context of special services.

1.10.3 Authorization ID

Identifies the object to be authorized, i.e. the name of the acronym or the ID of an application.

1.10.4 Authorization key

The authorization key that is used to protect the object.

1.10.5 Authorization Designation

(Optional) text description of the authorization.